

Utilizarea convertorului digital-analog si transferul de date intre periferice

1 Scopul lucrarii

Studentul ar trebui ca, la sfarsitul sedintei de laborator, sa aiba cunostintele de baza despre functionarea sistemului de conversie digital-analog al chip-ului, despre timer-ele incluse in SoC (System-on-Chip).

2 Introducere

In acest capitol vor fi prezentate:

- 2.1. Informatii aditionale despre modul de lucru al convertorului ADC2
- 2.2. Sistemul de conversie digital-analog
- 2.3. Timere
- 2.4. Transferul de date intre periferice

2.1 Informatii aditionale despre modul de lucru al convertorului ADC2

Convertorul ADC2 are la baza principiul functionarii unui registru de aproximari succesive (SAR), fiecare conversie fiind efectuata cu ajutorul unui ceas propriu, care deriva din ceasul sistemului. In functie de configurarea bitilor 7-3 ai registrului ADC2CF (AD2SC), se seteaza raportul de divizare intre ceasul sistemului si ceasul SAR-ului, dupa ecuatie:

$$CLK_{SAR2} = \frac{SYSCLK}{AD2SC + 1}$$

unde: SYSCLK este ceasul sistemului,

AD2SC este valoarea in zecimal a numarului format de bitii 7-3 ai registrului ADC2CF.

Cu toate acestea, frecventa maxima a SAR-ului nu poate depasi 7.5 MHz.

Convertorul poate urmari semnalul de la intrare in mod continuu (cum s-a lucrat pana acum), sau doar atunci cand are loc o conversie, in restul timpului el nefunctionand. Aceste moduri de lucru pot fi configurate cu ajutorul bitului AD2TM din registrul AD2CN:

- **modul de lucru continuous tracking (urmarire continua):** convertorul este mereu activ si urmareste semnalul de intrare, iar conversia poate fi inceputa imediat ce se primeste comanda (de exemplu setarea bitului AD2BUSY) si dureaza 8 tacte de ceas ale SAR-ului.
- **modul de lucru low-power (economic de putere):** atat timp cat dispozitivul nu face o conversie, el functioneaza intr-un mod low-power, in care consuma foarte putina putere. Acest mod de lucru este foarte eficient pentru sistemele in care se doreste un consum redus de putere. Totusi, la primirea unei comenzi de conversie, sunt necesare 3 tacte de ceas pentru ca dispozitivul sa poata incepe conversia. In acest caz, timpul necesar unei conversii creste de la 8 tacte la 11 tacte de ceas ale SAR-ului, dupa cum se poate observa in Fig. 2.1.

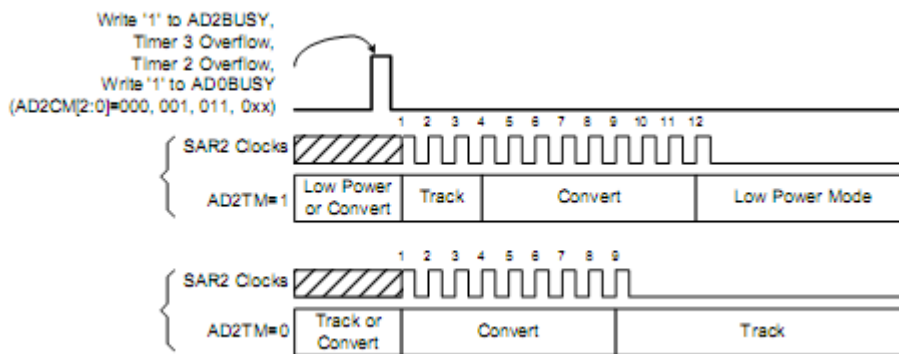


Fig. 2.1. Cronograma conversiei pentru modurile low power si continuous tracking

2.2 Sistemul de conversie digital-analog (DAC)

Microcontrolerul dispune de doua sisteme de conversie digital-analog, ambele pe 12 biti, numite DAC0 si DAC1. Acestea au acelasi mod de configurare, astfel ca informatiile prezentate in lucrarea de fata sunt valabile atat pentru DAC0, cat si pentru DAC1. Singura diferenta dintre ele este ca registrele corespunzatoare DAC0 se afla pe pagina 0, iar registrele corespunzatoare DAC1 se afla in pagina 1 a registrelor SFR. Intrucat programul foloseste DAC0, vom da detaliile de configurare pentru acest convertor.

In Fig. 2.1. este prezentata diagrama bloc a convertorului DAC0.

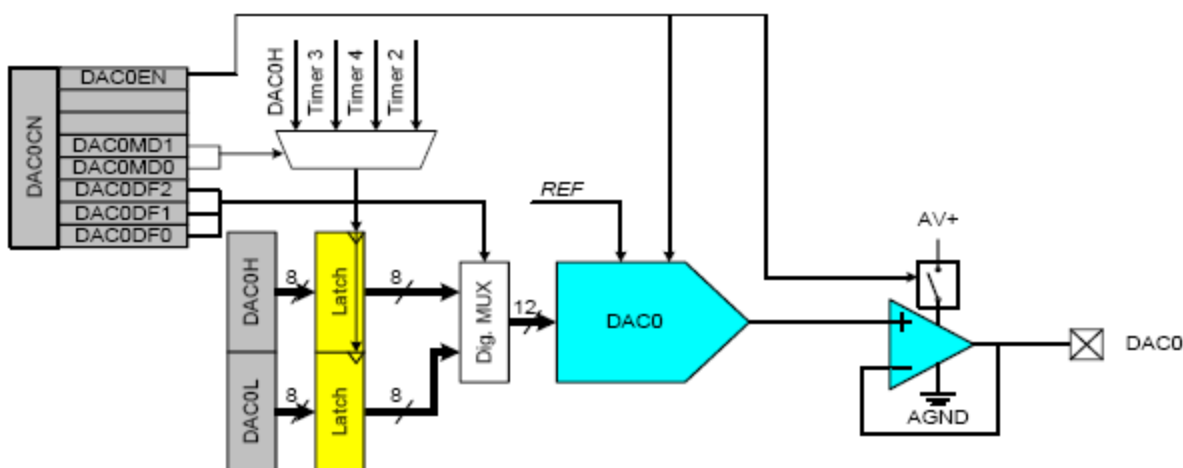


Fig. 2.2. Schema bloc a convertorului DAC0

Dispozitivul nu este activat implicit la resetarea microcontrolerului. Pentru pornirea lui, se va seta bitul 7 (DAC0EN) din registrul DAC0CN. Tensiunea de iesire pentru convertor variaza de la 0 la valoarea de pe pinul VREFD, ce este rutat direct la referinta de tensiune si poate avea valoarea de 1.2V sau 2.4V. Dupa cum se observa in schema bloc, convertorul are asociate doua registre (DAC0L si DAC0H) unde se scrie valoarea ce urmeaza a fi convertita.

2.2.1 Moduri de conversie ale DAC0

Convertorul poate face transformarea digital-analog in doua moduri:

- La cerere (on-demand): tensiunea de iesire este actualizata la scrierea in registrul DAC0H. Observatie: Valoarea scrisa in DAC0L este pastrata in registru, fara a fi trimisa in latch-ul corespunzator, pana cand are loc o scriere in registrul DAC0H. Pentru utilizarea pe 12 biti este necesar ca, mai intai, sa se scrie informatia in DAC0L, urmata de scrierea in DAC0H. Daca se doreste utilizarea convertorului doar pe 8 biti, este sugerat ca in DAC0L sa se scrie 0x00, si intregul octet necesar conversiei sa fie scris in DAC0H.
- Comandata de o depasire in timer: similar cu functionarea unui ADC, conversia DAC-ului poate fi initializata de o depasire a unui timer. In acest mod, scrierea in registrele de date ale convertorului poate fi facuta in orice ordine, deoarece valorile din registre nu sunt trecute in

latch-urile corespunzatoare decat atunci cand are loc un overflow al timerului. Selectarea timerului dupa care se face conversia se realizeaza cu ajutorul bitilor 4 si 3 din registrul DAC0CN.

2.2.2 Scalarea tensiunii de iesire a convertorului

In anumite cazuri, datele de intrare in convertor trebuie rearanjate inainte de scrierea lor. Aceasta operatie presupune instructiuni suplimentare, care maresc dimensiunea codului executabil si micsoreaza viteza DAC-ului. Pentru a inlatura acest neajuns, programatorul are posibilitatea sa modifice asezarea informatiei in cele doua registre de date ale convertorului, cu ajutorul bitilor 0, 1 si 2 din registrul de configurare DAC0CN. Valorile acestor biti corespunzatoare diferitelor asezari ale datelor sunt prezentate in Fig. 2.2.2. Pentru mai multe informatii despre definitia DAC0CN consultati documentatia microcontrolerului (fisierele C8051F04xRev1_4.pdf, pag. 108).

Bits2-0: DAC0DF2-0: DAC0 Data Format Bits:

000: The most significant nibble of the DAC0 Data Word is in DAC0H[3:0], while the least significant byte is in DAC0L.

DAC0H								DAC0L							
				MSB											LSB

001: The most significant 5-bits of the DAC0 Data Word is in DAC0H[4:0], while the least significant 7-bits are in DAC0L[7:1].

DAC0H								DAC0L							
				MSB										LSB	

010: The most significant 6-bits of the DAC0 Data Word is in DAC0H[5:0], while the least significant 6-bits are in DAC0L[7:2].

DAC0H								DAC0L							
				MSB										LSB	

011: The most significant 7-bits of the DAC0 Data Word is in DAC0H[6:0], while the least significant 5-bits are in DAC0L[7:3].

DAC0H								DAC0L							
							MSB							LSB	

1xx: The most significant 8-bits of the DAC0 Data Word is in DAC0H[7:0], while the least significant 4-bits are in DAC0L[7:4].

DAC0H								DAC0L							
							MSB							LSB	

Fig. 2.2.2. Modulile de asezare a datelor

2.3 Timer 3

Timer 3 este un timer/counter pe 16 biti implementat cu doua registre speciale astfel: TMR3L (stocheaza octetul inferior) si TMR3H (stocheaza octetul superior).

In functie de configuratia bitilor T3M1, T3M0 din registrul TMR3CF, timerul poate fi sincronizat cu una din urmatoarele surse de ceas:

- ceasul de sistem,
- ceasul de sistem cu frecventa divizata prin 2 sau 12,
- un oscilator extern cu frecventa divizata prin 8,
- fronturile cazatoare ale semnalului de pe pinul T3 (numai in modul capture).

Timerul poate functiona ca numarator, contorizand fronturile cazatoare ale unui semnal extern aplicat pe un pinul T3 (pentru aceasta bitul C/T3 din registrul TMR3CN trebuie setat). Timer-ul poate numara crescator sau descrescator (implicit Timer 3 numara crescator).

Timer 3 se activeaza setand bitul TR3 din registrul TMR3CN.

Bitul CP/RL3 din registrul TMR3CN stabileste modul de functionare al Timer 3:

- modul 0 – auto-reload
- modul 1 – capture

Pentru mai multe informatii despre definitia TMR3CN si definitia TMR3CF consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 301, pag. 302).

2.3.1 Modul de lucru auto-reload

Modul auto-reload se selecteaza resetand bitul CP/RL3 din registrul TMR3CN. In acest mod de functionare, Timer 3 poate fi setat sa numere crescator sau descrescator. Implicit, va numara crescator. In momentul unei depasiri (tranzitie 0xFFFF – 0x0000):

- registrele TMR3L si TMR3H se vor reincarca cu valorile stocate in registrele RCAP3L, respectiv RCAP3H. Numaratoarea nu reincepe de la 0x0000, ci de la valoarea stocata in aceste registre.
- bitul TF3 din registrul TMR3CN se seteaza. Daca intreruperile asociate acestui timer sunt activate, atunci setarea acestui bit va genera o intrerupere.
- bitul EXF3 din registrul TMR3CN isi schimba valoarea logica. Acest bit poate fi considerat cel mai semnificativ bit al unui numarator pe 17 biti.

Reincarcarea Timer 3 (a registrelor de numarare TMR3L si TMR3H) poate fi comandata si de utilizator. Pentru aceasta trebuie setat bitul EXEN3 din registrul TMR3CN si resetat bitul DCEN din registrul TMR3CF. Aceste configurari fiind facute, fiecare front cazator al semnalului aplicat de utilizator pe pinul T3EX va determina o reincarcare. Atentie: in cazul in care bitul DCEN are valoarea logica 1, Timer 3 va numara crescator, respectiv descrescator, daca semnalul aplicat pe pinul T3EX are valoarea logica 1, respectiv 0, iar tranzitiile acestui semnal nu vor determina reincarcari.

Pentru mai multe informatii despre definitia RCAP3H si definitia RCAP3H consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 303).

2.3.2 Modul de lucru capture

Modul capture se poate selecta setand bitul CP/RL3 din registrul TMR3CN. De asemenea, bitul EXEN3 din acelasi registru trebuie setat pentru a activa capturile. Acest mod este folosit daca se doreste stocarea valorii timerului la un anumit moment de timp specificat de utilizator. Cand Timer 3 este configurat in modul capture, fronturile cazatoare ale semnalului aplicat pinului T3EX determina:

- stocarea valorilor registrelor TMR3L si TMR3H in registrele RCAP3L, respectiv RCAP3H.
- setarea bitului EXF3 din registrul TMR3CN. Daca intreruperile asociate acestui timer sunt activate, atunci setarea acestui bit va genera o intrerupere.

Implicit Timer-ul numara in sus, insa sensul de numarare poate fi schimbat prin setarea bitului DCEN din registrul TMR3CF. O depasire (tranzitie 0xFFFF – 0x0000, la numararea in sus sau tranzitie 0x0000 – 0xFFFF, la numararea in jos) determina setarea bitului TF3 din registrul TMR3CN. Daca intreruperile asociate acestui timer sunt activate, atunci setarea acestui bit va genera o intrerupere.

Observatie: toate registrele de configurare, inclusiv cele de reincarcare si cele de numarare, se afla in pagina 1 a registrelor SFR.

3 Problema

3.1 Formularea problemei

Sa se implementeze un sistem care sa transfere semnalul vocal captat de un microfon la un difuzor.

3.2 Solutii posibile

3.2.1 Descrierea modulelor hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza placa de dezvoltare cu microcontrolerul C8051F040. Acesteia i se adauga doua module externe:

- modulul de intrare
 - rolul acestui modul este de a amplifica semnalul de la microfon, intrucat acesta are o amplitudine foarte mica (de ordinul milivoltilor) si trebuie protejat impotriva zgomotului.
 - contine un circuit de amplificare a semnalului audio de la microfon si un filtru trece jos.
 - se conecteaza la microcontroler prin doua fire; unul este masa sistemului, iar celalalt transmite semnalul audio de intrare.
 - schema modului de intrare este prezentata in Fig. 3.2.1.a.

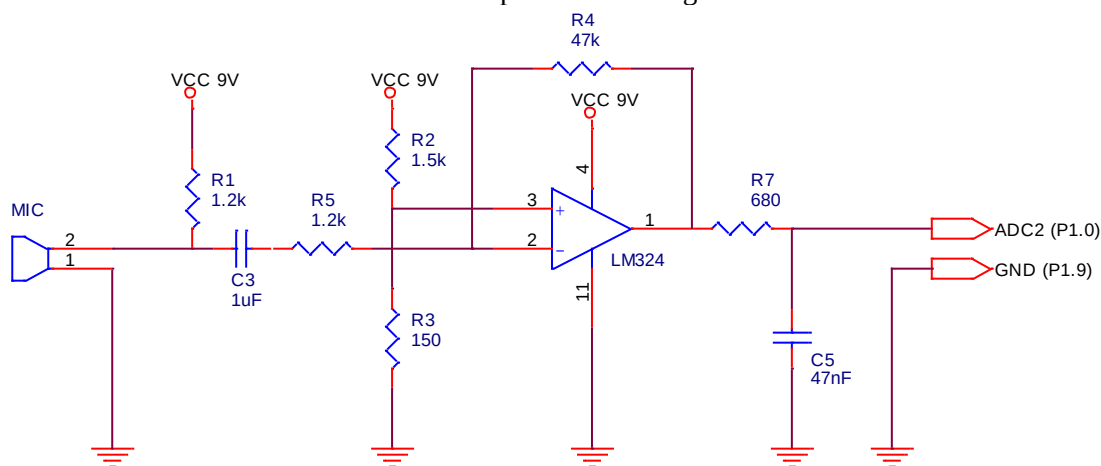


Fig. 3.2.1.a. Amplificatorul de intrare

Semnalele furnizate de un microfon uzual cu alimentare nu pot fi folosite direct de convertor intrucat tensiunea generata de acesta se plaseaza in domeniul milivoltilor. De asemenea, ADC2 nu convertește decat tensiuni pozitive. Din aceste motive, semnalul furnizat de microfon este trecut printr-un etaj de amplificare cu amplificator operational cu reactie negativa cu un castig fix.

Intrucat amplificatorul este alimentat de la o sursa unipolara va trebui ridicat nivelul de tensiune continua al semnalului vocal amplificat. Cum plaja de tensiuni convertite este intre 0V si 2.4V se va stabili nivelul de curent continuu la 1.2V pentru ca excursia semnalului vocal sa se incadreze intre limitele mai sus mentionate. Pe iesirea amplificatorului, conectam un filtru trece jos antialiere (format din rezistenta R7 si condensatorul C5) pentru a limita banda semnalului de intrare la 4 kHz. Mentionam ca vom folosi referinta de tensiune de 2.4V pentru a obtine un raport semnal util – zgomot cat mai mare.

- modulul de iesire
 - rolul acestui modul este de a creste nivelul curentului de la iesirea convertorului DAC0, astfel incat acesta sa poata comanda o sarcina de ordinul ohmilor sau zecilor de ohmi, tipica unor casti multimedia. In cazul de fata, impedanta castilor este de 32 ohmi.
 - modulul contine un amplificator operational de putere, cu o capacitate mare in curent, doua filtre trece-jos si un divizor rezistiv.
 - schema modului de iesire este prezentata in Fig. 3.2.1.b.

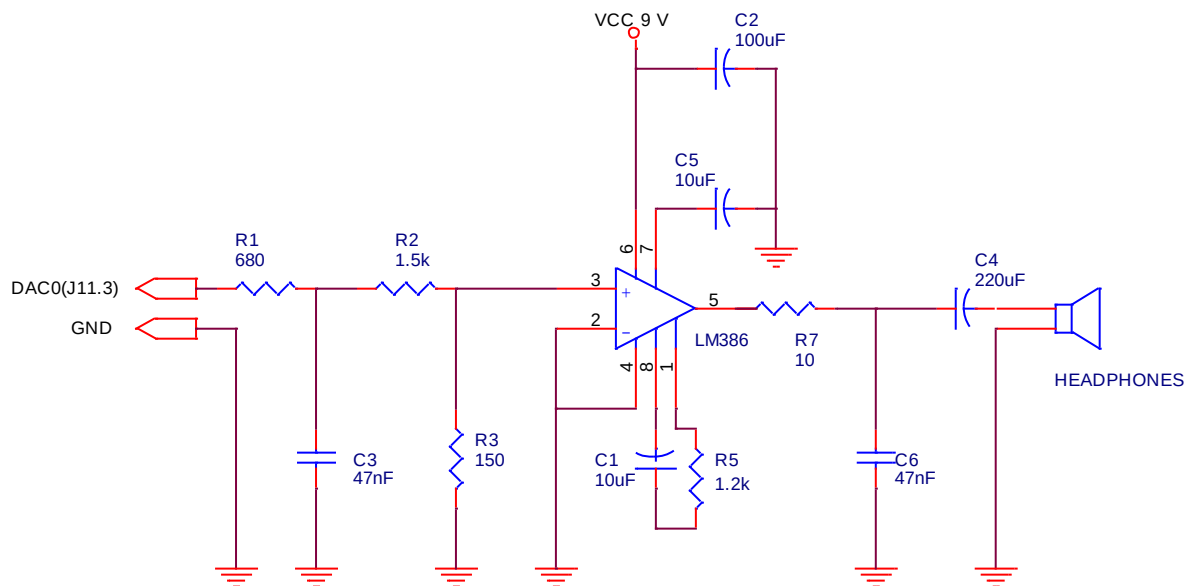


Fig 3.2.1.b. Amplificatorul de iesire

Unul dintre filtre (format din R1 si C3) este pus imediat dupa convertor, pentru a elimina zgomotul de cuantizare (pentru a elimina frecventele inalte si a atenua salturile bruste din semnalul furnizat de convertor), iar cel de-al doilea (format din R7 si C6) aflat dupa amplificator, pentru a diminua zgomotul propriu al acestuia. Divizorul rezistiv (R2 si R3) este folosit pentru a reduce amplitudinea semnalului de intrare si a elimina riscul ca iesirea amplificatorului sa fie adusa la tensiunea de alimentare

3.2.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2. In solutia propusa programul va avea doua rutine: main si init.

Algoritmul consta intr-o bucla ce se autoapeleaza la infinit. Aceasta bucla are doua etape:

- se convertește tensiunea furnizata de modulul de intrare intr-o valoare digitala pe 8 biti.
- rezultatul convertit este transferat in DAC0H (cei mai semnificativi 8 biti din totalul de 12 biti ai DAC0).

Rutina principala (main) apeleaza subrutina de initializare (init) apoi trece in zona de bucla infinita etichetata start. Zona etichetata start respecta etapele algoritmului de mai sus si apoi se reapeleaza.

Subrutina de initializare

- dezactiveaza intreruperile
 - pentru detalii vezi explicatiile din Lucrarea nr. 1.
- dezactiveaza watchdog timer-ul
 - pentru detalii vezi explicatiile din Lucrarea nr. 1.
- activeaza crossbar-ul
 - pentru detalii vezi explicatiile din Lucrarea nr. 1.
- configureaza portul P1
 - pentru detalii vezi explicatiile din Lucrarea nr 2.
- configureaza tensiunea de referinta
 - pentru detalii vezi explicatiile din Lucrarea nr 2.
- configureaza convertorul analog-digital
 - pentru detalii vezi explicatiile din Lucrarea nr 2.
- configureaza convertorul digital-analog
 - se activeaza convertorul setand bitul DAC0EN.
 - se configureaza momentul de update al tensiunii de iesire a DAC0 ca fiind momentul scrierii in registrul DAC0H.
 - se configureaza formatul datelor din DAC0 ca in ultima varianta din Fig. 2.2.2.

- pentru mai multe informatii despre definitia DAC0CN consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 108).

Observatie: Acest algoritm este foarte simplu, dar ofera foarte putin control asupra ratei de esantionare si asupra cantitatii de informatie transferata. Este de mentionat ca rata de esantionare a semnalului vocal analogic este determinata exclusiv de frecventa de ceas la care lucreaza registrul de aproximari succesive (SAR) prin aceasta determinandu-se efectiv durata unei conversii, precum si numarul de cicli de ceas ai sistemului, necesari pentru executia fiecărei instructiuni din bucla `start`.

Organigrama programului este prezentata in Fig. 3.2.2.a.

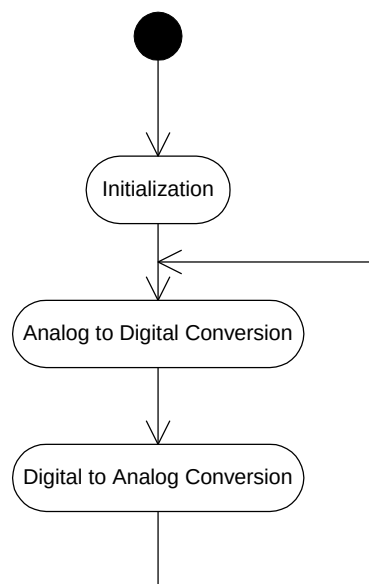


Fig. 3.2.2.a. Organigrama aplicatiei AD-DAConversion_v1

Pentru un control direct asupra cantitatii de informatie transferata, propunem o alta solutie in care conversia semnalului de la microfon si transferul catre casti se efectueaza periodic, cu o perioada bine determinata de o depasire a unui timer.

Structura programului se pastreaza, cu urmatoarele modificari:

- in bucla etichetata `start` se introduce o etapa suplimentara in care se asteapta o depasire a Timer 3
- in subrutina de initializare se configureaza si Timer 3
 - se configureaza modul auto-reload cu frecventa de incrementare egala cu frecventa ceasului (~ 3.06MHz)
 - pentru a obtine o frecventa de esantionare de 16KHz (s-a ales aceasta frecventa ca fiind mai mare decat frecventa limita de esantionare) se calculeaza valoarea de reload 0xFF44 (valoare ce se stocheaza in RCAP3H si RCAP3L) conform formulei:

$$F_s = \frac{F_{CK}}{65536}$$

Organigrama programului este prezentata in Fig. 3.2.2.b.

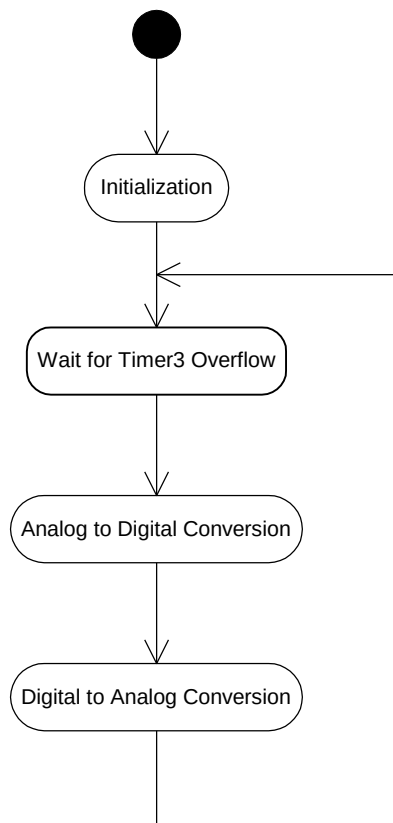


Fig. 3.2.2.b. Organigrama aplicatiei AD-DAConversion_v2

4 Desfasurarea lucrarii

1. Creati un nou proiect folosind fisierul Z:\Lab8051\Laborator4\AD-DACConversion_v1.asm urmand pasii prezentati in Lucrarea de laborator nr. 1, la capitolul 2.2.
2. Completati codul subrutinei de initializare in vederea dezactivarii intreruperilor, a watchdog timer-ului si activarii crossbar-ului, configurarii portului P1, tensiunii de referinta interna, convertorului analog-digital ADC2. Indicatie: urmariti explicatiile prezentate in Lucrarea nr. 2, la capitolul 2.1 si ANEXA 5.1.
3. Executati aplicatia apasand butonul  aflat in bara de butoane a IDE-ului si observati comportarea sistemului.
4. Creati un nou proiect folosind fisierul Z:\Lab8051\Laborator4\AD-DACConversion_v2.asm urmand pasii prezentati in Lucrarea de laborator nr. 1, la capitolul 2.2.
5. Completati codul subrutinei de initializare in vederea dezactivarii intreruperilor, a watchdog timer-ului si activarii crossbar-ului, configurarii portului P1, tensiunii de referinta interna, convertorului analog-digital ADC2. Indicatie: urmariti explicatiile prezentate in Lucrarea nr. 2, la capitolul 2.1 si ANEXA 5.1.
6. Executati aplicatia apasand butonul  aflat in bara de butoane a IDE-ului si observati comportarea sistemului.
7. Sa se modifice programul AD-DACConversion_v2.asm astfel incat esantionarea semnalului vocal sa se faca la limita conditiei Nyquist. Sa se calculeze in consecinta valorile ce trebuiesc introduse in RCAP3, sa se observe efectele subesantionarii.
8. Sa se editeze un program care sa genereze un semnal dreptunghiular.
9. Sa se editeze un program care sa genereze un semnal dinte de ferastrau.
10. Sa se editeze un program care sa genereze un semnal triunghiular.
11. Sa se editeze un program care sa modifice frecventa de stingere/aprindere a ledului in functie de amplitudinea semnalului vocal.
12. Sa se editeze un program care moduleaza amplitudinea luminozitatii ledului in functie de amplitudinea semnalului vocal.
13. Sa se editeze un program care, folosind tastatura sa genereze cu ajutorul convertorului DAC0 notele corespunzatoare gamei DO, conform urmatoarei corespondente: tasta a – nota DO de jos, tasta s – nota RE, tasta d – nota MI, tasta f – nota FA, tasta g – nota SOL, tasta h – nota LA, tasta j – nota SI si tasta k – nota DO de sus. Frecventele corespunzatoare fiecărei note se gasesc in ANEXA 5.3.
14. Sa se modifice programul AD-DACConversion_v1.asm astfel incat transferul sa se faca 12biti, fara sa se faca schimbarea paginii de SFR.
15. Sa se editeze un program prin care informatia de la ambele ADC-uri sa fie transferata unui singur DAC.

5 ANEXE

5.1 Codul aplicatiei AD-DAConversion_v1

```

;-----
;
;
; FILE NAME   : AD-DAConversion_v1.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : Data transfer from ADC2 to DAC0.
;
; NOTES:
;-----

        $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg        AT 0x0000            ; Reset Vector
                ljmp        main                ; Locate a jump to the start of code
                                                ;at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg      segment      CODE

                rseg        mainCodeSeg        ; Switch to this code segment.
                using       0                  ; Specify register bank for the following
                                                ; program code.

main:            acall       init                ; Initialization of all used SFR's

start:          ; Starting the algorithm
                mov         SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page
                clr         AD2INT              ; Reset AD2INT.
                setb        AD2BUSY             ; Force ADC2 to start a new conversion
                jnb         AD2INT, $           ; Wait for ADC2 to finish the conversion
                mov         A, ADC2             ; Write the converted value in the accumulator

                mov         SFRPAGE, #DAC0_PAGE ; Use SFRs on the DAC0 Page
                mov         DAC0H, A            ; Write the value from the accumulator
                                                ;in upper byte of DAC0
                sjmp        start               ; Start over again

;-----
; FUNCTION CODE
;-----

init:           ...                ; Disable global interrupts
                ...                ; Disable Watch Dog Timer

initIOandCross: ...                ; Use SFRs on the configuration Page

                ...                ; Enable Crossbar
                ...                ; Configure P1.0 as analog input
                ...                ; For analog data the port must be set as
                ...                ; open-drain.

initVREF:       ...                ; Use SFRs on the 0x00 Page
                ...                ; ADC2 voltage reference from internal VREF
                ...                ; Enable Bias Generator

initADC2:       ...                ; Use SFRs on the ADC2 Page
                ...                ; Set AIN1.0 as single-ended input
                ...                ; Select AIN1.0 as input channel for
                ...                ; the analog multiplexer
                ...                ; Configure a *1 Gain
                ...                ; Enable ADC2; Continuous tracking;

```

```
initDAC0:      mov     SFRPAGE, #DAC0_PAGE      ; Use SFRs on the DAC0 Page
               mov     DAC0CN, #0x84           ; Enable DAC0
               ;a conversion will start only after a write
               mov     DAC0L, #0x00           ;in DAC0H; the data format used is lxx.

               ret

;-----
; End of file.

END
```

5.2 Codul aplicatiei AD-DAConversion_v2

```
;-----
;
;
; FILE NAME   : AD-DAConversion_v2.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : Data transfer from ADC2 to DAC0.
;
; NOTES:
;
;-----

        $include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

               cseg      AT 0x0000                ; Reset Vector
               ljmp      main                    ; Locate a jump to the start of code
                                               ;at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg      segment      CODE

               rseg      mainCodeSeg            ; Switch to this code segment.
               using     0                      ; Specify register bank for the following
                                               ; program code.

main:            acall     init                  ; Initialization of all used SFR's

start:           ; Starting the algorithm
               mov     SFRPAGE, #TMR3_PAGE      ; Use SFRs on the Timer3 Page
               jnb     TF3, $                   ; Waiting for a Timer3 overflow.
               clr     TF3                      ; Reset the overflow flag.

               mov     SFRPAGE, #ADC2_PAGE      ; Use SFRs on the ADC2 Page
               clr     AD2INT                   ; Reset AD2INT.
               setb    AD2BUSY                  ; Force ADC2 to start a new conversion
               jnb     AD2INT, $                 ; Wait for ADC2 to finish the conversion
               mov     A, ADC2                  ; Write the converted value in the accumulator

               mov     SFRPAGE, #DAC0_PAGE      ; Use SFRs on the DAC0 Page
               mov     DAC0H, A                 ; Write the value from the accumulator
                                               ;in upper byte of DAC0
               sjmp     start                   ; Start over again

;-----
; FUNCTION CODE
;-----

init:           ...                            ; Disable global interrupts
               ...                            ; Disable Watch Dog Timer

initIOandCross: ...                            ; Use SFRs on the configuration Page

               ...                            ; Enable Crossbar
               ...                            ; Configure P1.0 as analog input
               ...                            ; For analog data the port must be set as
               ...                            ; open-drain.
```

```
initVREF:      ...      ; Use SFRs on the 0x00 Page
               ...      ; ADC2 voltage reference from internal VREF
               ...      ; Enable Bias Generator

initADC2:      ...      ; Use SFRs on the ADC2 Page
               ...      ; Set AIN1.0 as single-ended input
               ...      ; Select AIN1.0 as input channel for
               ...      ; the analog multiplexer
               ...      ; Configure a *1 Gain
               ...      ; Enable ADC2; Continuous tracking;

initDAC0:      mov      SFRPAGE, #DAC0_PAGE      ; Use SFRs on the DAC0 Page
               mov      DAC0CN, #0x84      ; Enable DAC0
               ;a conversion will start only after a write
               mov      DAC0L, #0x00      ;in DAC0H; the data format used is 1xx.

initTimer3:    mov      SFRPAGE, #TMR3_PAGE      ; Use SFRs on the Timer3 Page
               mov      TMR3CF, #0x08      ; Counting frequency is the SYSCLK frequency
               mov      RCAP3H, #0xFF      ; Set the reload value
               mov      RCAP3L, #0x44      ; The reload value triggers a sampling
               mov      TMR3H, #0xFF      ; frequency of 16kHz. See the formula!
               mov      TMR3L, #0x44
               mov      TMR3CN, #0x04      ; Enable Timer3.

               ret

;-----
; End of file.

END
```

5.3 Frecventele corepunzatoare notelor din gama Do

Pianul are opt note intr-o octava, fiecare nota fiind generata la apasarea uneia din tastele a, s, d, f, g, h, j, k. Notele au urmatoarele frecvențe:

Do – 55Hz
Re – 61.735Hz
Mi – 65.406Hz
Fa – 73.416Hz
Sol – 82.407Hz
La – 87.307Hz
Si – 97.999Hz
Do – 110Hz